

Fringe Projection Project

Implementation Roadmap

From notebook to working pipeline — step by step

What this document is

A staged plan for building the fringe projection metrology system in Python. Designed for the current state: working theoretical understanding, existing notebook with validated math, and limited hardware capability while waiting for mounting hardware and the upgraded projector.

How to use it

Work through the stages in order. Each stage has clear goals and exit criteria. Stages 1–4 don't need hardware. Stages 5–6 are when real components come in. Items marked with the "hardware optional" tag can be done now even without rigid mounting.

Tools you have available

Tool	Use in this project
Python 3.10	Main development environment (preferred over 3.14)
VS Code + Claude Code extension	Primary IDE for refactoring and module development
NumPy, OpenCV	Math core, image handling
Git	Version control (highly recommended from day 1)
Node.js + npm	Already used to install Claude Code
MATLAB	Cross-validation of Python results against reference
Fiji/ImageJ	Quick visual inspection of fringe images
MeshLab + CloudCompare	View 3D height maps and point clouds (later)
GitHub CLI + Miniconda	Coming soon from IT — useful but not blocking
Spinnaker SDK + PySpin	ADD TO IT LIST — needed for FLIR camera (later)

Stage 0 — Project Setup

Goal: A clean working environment, version control, and an organized project folder.

Time estimate: 1–2 hours

Hardware: None

0.1 — Create the project directory

Pick a location on your laptop and create the structure below. Use VS Code to manage it.

```
fringe_projection_project/
■■■ PROJECT_CONTEXT.md          # context for Claude Code
■■■ CONVERSATION_SUMMARY.md     # detailed prior context
■■■ reference/
■   ■■■ CHAPTER2_*.pdf          # theory chapter
■   ■■■ Chapter4_*.pdf         # practical chapter
■■■ notebooks/
■   ■■■ Fringe_Projection_Python.ipynb
■■■ src/                        # modules go here (Stage 2+)
■■■ tests/                      # tests for math (Stage 2+)
■■■ data/                      # synthetic fringes, calibration files
■■■ docs/
■   ■■■ Projector_Geometry_Summary.docx
■■■ README.md
```

0.2 — Initialize Git

```
cd fringe_projection_project
git init
git add .
git commit -m "Initial project structure"
```

Commit early, commit often. Even before code, commit docs and PDFs so you have history. When GitHub CLI is available, push to a private repo.

0.3 — Set up a Python environment

If you have Miniconda: `conda create -n fringe python=3.10`, then `conda activate fringe`.

If not, use venv: `python -m venv venv`, then `venv\Scripts\activate` (Windows).

Install the libraries you'll need:

```
pip install numpy scipy scikit-image opencv-python matplotlib jupyter pyqt6
```

0.4 — Confirm Claude Code works

Open VS Code in the project directory. Open Claude Code. Tell it: *"Read PROJECT_CONTEXT.md and CONVERSATION_SUMMARY.md, then summarize back what we're building."* Confirm it has the right context before moving on.

Stage 0 exit criteria

Project directory exists with reference materials inside. Git is initialized with a first commit. Python environment is active and core libraries import without errors. Claude Code can read your context files.

Stage 1 — Close the Simulation Loop

Goal: Add the missing physical projection step to the existing notebook and validate the inverse grating method end-to-end in simulation.

Time estimate: 2–4 hours

Hardware: None

Why this matters before refactoring

Your existing notebook computes ϕ_{sim} correctly but doesn't simulate the full physical loop (project pattern → projector distorts → camera captures). Until you close this loop, you can't actually verify that your inverse grating cancels bias. Doing this in the existing notebook is faster than refactoring first.

1.1 — Add the projection function

In your notebook, after the system parameters cell, add:

```
def project(input_phase):  
    """Simulates non-telecentric projector adding perspective bias."""  
    bias = (2 * np.pi / p1) * (X**2 * np.tan(theta) / a)  
    return input_phase - bias
```

1.2 — Validate against your existing ϕ_{sim}

Confirm the function works:

```
uniform_phase = (2 * np.pi / p1) * X  
phi1_simulated = project(uniform_phase)  
# Should match the original phi1 in your notebook  
print(np.allclose(phi1_simulated, phi1))
```

1.3 — Validate the inverse grating cancels bias

Project the inverse pattern ϕ_{inv} through the same biased system:

```
result = project(phi2)  
# Should be a clean uniform ramp: (2*pi/p1) * X  
expected = (2 * np.pi / p1) * X  
residual = result - expected  
print('Residual after correction:', residual.std())
```

If the residual is essentially zero (within numerical noise), the inverse grating method works.

1.4 — Document and commit

Add a markdown cell in the notebook explaining what was added and why. Commit to Git with a message like: "Close inverse-grating simulation loop with project() function".

Stage 1 exit criteria

The notebook contains a `project()` function that adds perspective bias. Projecting a uniform phase through it reproduces the original biased $\phi_{\text{■}}$. Projecting $\phi_{\text{■}}$ through it produces a clean ramp with negligible residual. The math is validated end-to-end.

Stage 2 — Refactor Notebook into Python Modules

Goal: Convert the working notebook into a clean, importable Python package.

Time estimate: 1–2 days

Hardware: None

Tool: Claude Code (this is exactly the kind of task it's good at)

2.1 — Plan the module structure

Tell Claude Code to create the following layout under `src/`:

Module	Contents
<code>geometry.py</code>	System parameters, Geometry class with HybridGeometry implementation
<code>pattern_generator.py</code>	Standard sinusoidal grating + inverse grating from ϕ
<code>synthetic_fringes.py</code>	Forward model: <code>project()</code> function, <code>generate_frames()</code>
<code>phase_shifting.py</code>	4-step and 8-step phase extraction
<code>unwrapping.py</code>	Wrap <code>np.unwrap</code> and <code>skimage.restoration.unwrap_phase</code>
<code>calibration.py</code>	Tilt-flip trick, bias map calculation, λ_{eq} computation
<code>reconstruction.py</code>	Phase $\rightarrow$ height conversion
<code>io_utils.py</code>	Save/load patterns, frames, calibration files

2.2 — Suggested order of refactoring

Don't ask Claude Code to do everything at once. Walk through one module at a time. After each, run a small test that proves the module reproduces the notebook's behavior.

- Refactor cells 1–2 $\rightarrow$ `geometry.py` + `synthetic_fringes.py` (`project`)
- Refactor cell 3 $\rightarrow$ `pattern_generator.py` (standard) + `synthetic_fringes.py` (`generate_frames`)
- Refactor cells 5–7 $\rightarrow$ `phase_shifting.py` + `unwrapping.py`
- Refactor cells 8–11 $\rightarrow$ `calibration.py` (tilt-flip)
- Refactor cell 18 $\rightarrow$ `reconstruction.py`
- Add `pattern_generator.py` (inverse) using ϕ from calibration

2.3 — Add a test script

Create `tests/test_pipeline_synthetic.py` that runs the entire pipeline end-to-end and asserts the recovered Gaussian matches the input within tolerance. This is your regression test — if any module breaks later, this will catch it.

2.4 — Document each module

Every function gets a docstring with: (a) what it does, (b) input/output shapes and units, (c) the chapter equation it implements (e.g., # Implements Eq. 4-7 from Samara Chapter 4).

Stage 2 exit criteria

All notebook logic lives in importable modules. The test script reproduces the notebook's result (recovered Gaussian within $\sim 1e-4$). The notebook becomes a thin demo that imports from src/.

Stage 3 — Upgrade the Forward Model

Goal: Replace the Taylor approximation with the exact Eq. 2-44 form so the simulation faithfully matches what real hardware will produce.

Time estimate: Half a day

Hardware: None

Cell 2 of the notebook uses a Taylor approximation: a quadratic-in- x bias term. The chapter's Eq. 2-44 has a more accurate form with a $1/[1 - 2 \cdot x \cdot \tan(\theta)/a]$ denominator. For small x this matches the Taylor form, but at the edges of the field of view they diverge.

3.1 — Add the exact forward model

In `synthetic_fringes.py`, add a parameter `model='taylor'` or `'exact'` to the `project()` function. Implement the exact Eq. 2-44 version and let the user choose.

3.2 — Compare both models

Generate frames with both models and check the difference. For your FOV (~68 mm) the difference may matter at the edges.

3.3 — Test the inverse grating still works

Re-run the validation from Stage 1 with the exact model. The tilt-flip trick is supposed to work for any bias shape, so it should still cancel correctly.

Stage 3 exit criteria

Forward model has both Taylor and exact options. Inverse grating still cancels bias under the exact model. Test pipeline still passes.

Stage 4 — Build the GUI Skeleton

Goal: A working PyQt6 user interface that drives the math modules with mock hardware.

Time estimate: 3–5 days

Hardware: None

4.1 — Tab structure

Tab	Purpose
Pattern	Choose pattern type, set parameters, preview, save/send to projector
Camera	Live preview (mock), capture sequence, save raw frames
Calibration	Acquire flat-reference data, compute ϕ , save calibration
Measurement	Load calibration, capture object, process, display 3D height map
Settings	System parameters, file paths, defaults

4.2 — Hardware mock layer

Define abstract Camera and Projector classes. Provide MockCamera (returns synthetic frames via the forward model) and MockProjector (saves PNGs / shows preview window). The GUI calls these abstractions, not real hardware. When real hardware arrives, swap implementations.

4.3 — End-to-end demo

By the end of Stage 4, you should be able to: open the GUI, click "Calibrate" (uses mock flat surface), click "Measure" (uses mock Gaussian object), see the recovered 3D height map appear. Without ever touching real hardware.

4.4 — Commit milestones

After each tab works, commit. Tag a version v0.1-gui-mock when all tabs work end-to-end with simulated data.

Stage 4 exit criteria

GUI runs and produces a recovered height map from synthetic data, end-to-end, in real time. All hardware access goes through Mock classes. Code is commit-clean.

Stage 5 — Hardware Familiarization (Optional, Parallel)

Goal: Get hands-on with the camera and projector. Learn the SDKs and quirks before they become blockers.

Time estimate: 1–2 days, in pieces

Hardware: Camera + projector (no mounting needed)

Why "optional"

Real measurements with this hardware aren't useful yet — no mounting, the Pico Genie is a placeholder, and proper calibration needs rigid alignment. But "hello world" experiments are still valuable to learn the tools without time pressure.

5.1 — Install Spinnaker SDK

Add to IT install list. Once installed, install PySpin in your Python environment (per FLIR's instructions).

5.2 — Capture one frame

Write a 10-line script that initializes the FLIR camera, captures a frame, and saves it as a PNG.

```
import PySpin
system = PySpin.System.GetInstance()
cam = system.GetCameras()[0]
cam.Init()
cam.BeginAcquisition()
image = cam.GetNextImage()
image.Save('test.png')
cam.EndAcquisition()
cam.DeInit()
system.ReleaseInstance()
```

5.3 — Project a pattern

Write a script that creates a sinusoidal grating image and displays it fullscreen on the extended display (the projector). Options: PIL.Image + tkinter, or pygame. Keep it simple.

5.4 — Combine: capture a fringe

Project the grating and capture it with the camera. Save the captured fringe image. Inspect in Fiji/ImageJ to see what real fringes look like.

5.5 — Camera settings exploration

Play with exposure, gain, framerate. Find a working baseline for typical room lighting and projector brightness. Note the values down — you'll need them for actual measurements.

Stage 5 exit criteria

You can capture a real fringe image from the camera at the press of a button. You know roughly what exposure/gain settings work. When mounting hardware arrives, you're ready to swap MockCamera for RealCamera.

Stage 6 — Real Hardware Integration

Goal: Replace the mock layer with real hardware. Run actual measurements.

Time estimate: 1–2 weeks

Hardware: Camera, upgraded projector, mounting hardware, calibration artifacts

This stage is for when all the hardware is in place. The roadmap below summarizes what to do but specifics will depend on the upgraded projector's properties and the mounting setup.

6.1 — Real Camera + Real Projector classes

Implement the same interface as MockCamera/MockProjector but with PySpin and fullscreen display. Test that GUI tabs work the same with real classes substituted in.

6.2 — Bias calibration

Procedure from Chapter 4 §4.4.1:

- Place flat reference (gauge block / optical flat) in field of view
- Project uniform phase-shifted fringes (8 frames)
- Capture, extract phase ϕ , apply tilt-flip $\rightarrow \phi$
- Repeat 50× with phase offsets and surface position changes (chapter procedure)
- Average $\rightarrow$ save bias map

6.3 — Vertical (height) calibration

Use a known step-height standard (gauge block stack or precision step). Compute λ_{eq} from measured step in waves vs. nominal height.

6.4 — Lateral calibration

Image a calibrated grid scale. Compute $\mu\text{m}/\text{pixel}$.

6.5 — First real measurement

Place a known test object (e.g., a calibration sphere, a step block, a coin). Run the full GUI workflow: Calibrate $\rightarrow$ Measure $\rightarrow$ display height map $\rightarrow$ export. Validate against known dimensions.

Stage 6 exit criteria

GUI produces real height maps of test objects. Vertical and lateral calibration data are saved. A demonstration measurement (e.g., a step block) recovers the known height to within expected precision.

Quick References

Decision: when to start hardware integration

You have...	Recommended action
No mounting + Pico Genie only (now)	Stage 1–4 (simulation), optional Stage 5 hardware familiarization
No mounting + upgraded projector arrives	Add Stage 5 quick test with new projector, but don't try to calibrate yet
Mounting hardware + Pico Genie	Wait for upgraded projector — calibrating Pico Genie is wasted work
Mounting hardware + upgraded projector	Start Stage 6 — full hardware integration

Open questions for supervisor

Don't block on these — proceed on best assumptions and ask in parallel.

- Is the upgraded projector telecentric? (Affects whether inverse grating is needed at all.)
- Use software post-correction (subtract bias) or hardware pre-correction (project inverse)?
- How many phase-shift steps? (Notebook uses 4; chapter uses 8.)
- What calibration artifacts (flat reference, step gauge, lateral scale) are available in the lab?
- GUI framework preference?

Key chapter equations

Equation	Used for
2-44	Forward model — full intensity equation with perspective bias
2-46 / 2-47	Phase $\rightarrow$ height + λ_{eq} (general non-telecentric)
2-52	λ_{eq} for symmetric telecentric (compare/sanity check)
2-54	Inverse grating period (theoretical reference)
2-57	Phase $\rightarrow$ height with inverse grating
4-2 to 4-7	Tilt-flip trick — operational core of the method
4-9 / 4-10	Clean phase on object after bias correction

This Week's Concrete To-Do

Practical, no waiting. Things you can start *today*:

#	Task	Time
1	Create project directory structure (Stage 0.1)	20 min
2	Initialize Git, commit reference materials (Stage 0.2)	10 min
3	Set up Python venv (or wait for Miniconda) and install dependencies (Stage 0.3)	15 min
4	Open Claude Code in the project directory and confirm it reads context files (Stage 0.4)	10 min
5	Add the project() function to the notebook (Stage 1.1)	10 min
6	Validate inverse grating cancels bias in simulation (Stage 1.2 + 1.3)	30 min
7	Commit the validation. End of day: math is bulletproof.	5 min

What success this week looks like

By end of week: you've validated the inverse grating method end-to-end in simulation, you've started refactoring with Claude Code, and you have one or two modules cleanly extracted from the notebook. No hardware needed. No blockers.

What this gives you that's tangible

A working repo, a validated algorithm, and the start of a clean codebase that your prof can see, your collaborator (Three.js simulation) can use, and that smoothly grows into the full GUI + hardware system. Each stage produces something useful even if the next stage is delayed by hardware.